

1.Average Sale Price

```
import numpy as np
house_data = np.array([
    [3, 1400, 7500000],
    [5, 2200, 12500000],
    [4, 1800, 9800000],
    [6, 3000, 18500000],
    [2, 1100, 5200000]
])
filtered = house_data[house_data[:, 0] > 4]
average_price = np.mean(filtered[:, 2])
print("Houses with more than 4 bedrooms:")
print(filtered)
print("\nAverage Sale Price =", average_price)
```

Houses with more than 4 bedrooms:

```
[[ 5 2200 12500000]
 [ 6 3000 18500000]]
```

Average Sale Price = 15500000.0

2.Logistic Regression

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
data = {
    "Num_Links": [1, 6, 0, 4, 2],
    "Num_Caps_Words": [3, 15, 1, 10, 5],
    "Email_Length": [120, 300, 80, 250, 160],
    "Spam": [0, 1, 0, 1, 0]
}
df = pd.DataFrame(data)
print("Training Dataset:\n")
print(df)
X = df[["Num_Links", "Num_Caps_Words", "Email_Length"]]
y = df["Spam"]
model = LogisticRegression()
model.fit(X, y)
new_email = [[7, 18, 350]]
prediction = model.predict(new_email)
print("\nNew Email Features:")
print(new_email)
if prediction[0] == 1:
    print("\nPrediction: Spam Email")
else:
    print("\nPrediction: Not Spam Email")
```

Training Dataset:

	Num_Links	Num_Caps_Words	Email_Length	Spam
0	1	3	120	0
1	6	15	300	1
2	0	1	80	0
3	4	10	250	1
4	2	5	160	0

New Email Features:
[[7, 18, 350]]

Prediction: Spam Email
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, warnings.warn(

3.Logistic Regression Spam Detection

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
data = {
    "Num_Links": [1, 6, 0, 4],
    "Num_Caps_Words": [3, 15, 1, 10],
    "Email_Length": [120, 300, 80, 250],
    "Spam": [0, 1, 0, 1]
}
df = pd.DataFrame(data)
print("Training Data:")
print(df)
X = df[["Num_Links", "Num_Caps_Words", "Email_Length"]]
```

```

y = df["Spam"]
model = LogisticRegression()
model.fit(X, y)
new_email = [[6, 15, 300]]
result = model.predict(new_email)
print("\nNew Email:", new_email)
if result[0] == 1:
    print("Prediction: Spam Email")
else:
    print("Prediction: Not Spam Email")

```

Training Data:

	Num_Links	Num_Caps_Words	Email_Length	Spam
0	1	3	120	0
1	6	15	300	1
2	0	1	80	0
3	4	10	250	1

New Email: [[6, 15, 300]]

Prediction: Spam Email

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, warnings.warn(

4.Linear Regression for Car Resale Price Prediction

```

import pandas as pd
from sklearn.linear_model import LinearRegression
data = {
    "Age": [2, 5, 3, 7, 1],
    "Kms_Driven": [25000, 60000, 40000, 90000, 15000],
    "Engine_CC": [1200, 1500, 1300, 1800, 1000],
    "Resale_Price": [650000, 420000, 520000, 300000, 720000]
}
df = pd.DataFrame(data)
print("Training Dataset:\n")
print(df)
X = df[["Age", "Kms_Driven", "Engine_CC"]]
y = df["Resale_Price"]
model = LinearRegression()
model.fit(X, y)
age = int(input("\nEnter Age of Car (years): "))
kms = int(input("Enter Kilometers Driven: "))
engine = int(input("Enter Engine CC: "))
prediction = model.predict([age, kms, engine])
print("\nPredicted Resale Price: ₹{:.2f}".format(prediction[0]))

```

Training Dataset:

	Age	Kms_Driven	Engine_CC	Resale_Price
0	2	25000	1200	650000
1	5	60000	1500	420000
2	3	40000	1300	520000
3	7	90000	1800	300000
4	1	15000	1000	720000

Enter Age of Car (years): 4

Enter Kilometers Driven: 190

Enter Engine CC: 1400

Predicted Resale Price: ₹537208.42

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, warnings.warn(

5. Loan Approval using CART (Decision Tree Classifier).

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
data = {
    "Income": [45000, 30000, 60000, 28000, 52000],
    "Credit_Score": [720, 650, 780, 620, 710],
    "Existing_Loans": [0, 1, 0, 2, 1],
    "Approved": [1, 0, 1, 0, 1]
}
df = pd.DataFrame(data)
print("Training Dataset:\n")
print(df)
X = df[["Income", "Credit_Score", "Existing_Loans"]]
y = df["Approved"]
model = DecisionTreeClassifier(random_state=0)
model.fit(X, y)

```

```
income = int(input("\nEnter Monthly Income (₹): "))
credit = int(input("Enter Credit Score: "))
loans = int(input("Enter Number of Existing Loans: "))
prediction = model.predict([[income, credit, loans]])
if prediction[0] == 1:
    print("\nLoan Status: Approved")
else:
    print("\nLoan Status: Rejected")
print("\nDecision Tree Rules:\n")
print(export_text(model, feature_names=list(X.columns)))
```

Training Dataset:

	Income	Credit_Score	Existing_Loans	Approved
0	45000	720	0	1
1	30000	650	1	0
2	60000	780	0	1
3	28000	620	2	0
4	52000	710	1	1

Enter Monthly Income (₹): 70000
Enter Credit Score: 250
Enter Number of Existing Loans: 1

Loan Status: Rejected

Decision Tree Rules:

```
|--- Credit_Score <= 680.00
|   |--- class: 0
|--- Credit_Score > 680.00
|   |--- class: 1
```

/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, warnings.warn(